

# Memoria Ascension | Videojuegos 2025-2026

---

## Grupo 1:

Nicolás Pascual 841142

Enrique Martínez 839773

Jorge Rodilla 845389

Jorge Pérez 821011

## Grupo 4:

Daniel Girón 870196

Óscar Jaime 818961



# Índice

|                                                                    |           |
|--------------------------------------------------------------------|-----------|
| <b>1. Visión general</b>                                           | <b>3</b>  |
| <b>2. Mecánicas del juego</b>                                      | <b>5</b>  |
| 2.1 Sala inicial                                                   | 5         |
| 2.2 Salas del Roguelike                                            | 7         |
| 2.3 Partido                                                        | 15        |
| 2.4 Tutoriales                                                     | 21        |
| <b>3. Gráficos</b>                                                 | <b>22</b> |
| 3.1 Salas de la liga                                               | 22        |
| 3.2 Partidos                                                       | 24        |
| <b>4. Inteligencia Artificial</b>                                  | <b>26</b> |
| 4.1 Escalado                                                       | 26        |
| 4.2 Comportamiento                                                 | 26        |
| 4.3 Portero                                                        | 28        |
| 4.4 Ayudas arbitrales                                              | 29        |
| <b>5. Generación del mapa</b>                                      | <b>30</b> |
| 5.1 Alternativa de diseño                                          | 31        |
| 5.2 Superposición de salas                                         | 31        |
| 5.3 Posición teórica de la sala inicial                            | 32        |
| 5.4 Sala del boss                                                  | 33        |
| <b>6. Sonido y música</b>                                          | <b>34</b> |
| 6.1 Estilo musical                                                 | 34        |
| 6.2 Uso del sonido                                                 | 34        |
| <b>7. Aspectos técnicos</b>                                        | <b>35</b> |
| 7.1 Motor                                                          | 35        |
| 7.2 Lenguaje                                                       | 35        |
| 7.3 Arquitectura                                                   | 35        |
| 7.4 Guardado                                                       | 35        |
| 7.5 Configuración y build                                          | 35        |
| <b>8. Apartado técnico 3D</b>                                      | <b>36</b> |
| 8.1 Modelos 3D                                                     | 36        |
| 8.2 Animaciones                                                    | 37        |
| 8.3 Mecánicas de disparo y paradas                                 | 38        |
| 8.4 Integración SFML + OpenGL                                      | 39        |
| <b>9. Organización del proyecto</b>                                | <b>40</b> |
| 9.1 Tareas y distribución                                          | 40        |
| 9.2 Cronograma                                                     | 42        |
| <b>10. Problemas encontrados</b>                                   | <b>43</b> |
| 10.1 Falta de assets e inconsistencia del arte                     | 43        |
| 10.2 Inteligencia artificial (IA) en la simulación de fútbol       | 43        |
| 10.3 Implementación y renderizado 3D en bajo nivel (OpenGL)        | 43        |
| 10.4 Modularidad para modificar número de jugadores                | 43        |
| 10.5 Compatibilidad multiplataforma (Linux - Windows)              | 44        |
| 10.6 Soporte para todo tipo de controles (Mando y teclado + ratón) | 44        |
| 10.7 Coordinación en el equipo                                     | 45        |
| 10.8 Integración de elementos Roguelike en juego de fútbol         | 45        |
| <b>11. Cumplimiento de objetivos</b>                               | <b>46</b> |
| <b>12. Mejoras después de la beta</b>                              | <b>47</b> |
| <b>13. Conclusión y cosas a futuro</b>                             | <b>48</b> |

# 1. Visión general

**Ascension** es un juego roguelike con temática de fútbol para un solo jugador, donde el jugador se desplaza por un mapa generado aleatoriamente con distintas salas en las que obtener objetos y mejoras a través de distintas mecánicas, y jugar partidos de fútbol contra distintos rivales para avanzar en el juego. En los partidos, el jugador controla un equipo reducido formado por tres jugadores de campo y un portero, y se enfrenta a un equipo rival con la misma composición controlado por IA.

El objetivo del juego es **ascender desde 1ª Federación hasta Primera División** y ganar el campeonato, completando un total de tres ligas por partida. Cada liga consta de un mapa aleatorio con un número creciente de partidos que hay que ganar para desbloquear la sala del jefe final, que es un partido con condiciones especiales que hay que superar para pasar a la siguiente liga. El juego termina cuando se gana el partido final de la última liga saltan los créditos y se vuelve al menú inicial, permitiendo iniciar una nueva partida.

Ha sido desarrollado para PC, pudiendo jugar usando el **teclado y ratón o el mando PlayStation 4**.

El juego está dirigido principalmente a jugadores casuales que disfrutan de experiencias de partidas cortas, alta rejugabilidad y progresión constante, propias de los roguelike. También resulta atractivo para aficionados al fútbol que buscan experiencias más arcade al contar con partidos más cortos y con menos jugadores que los simuladores habituales.

Nos basamos en juegos como **Nintendo World Cup (NES)** para la parte de los partidos y juegos estilo roguelike, como **Soul Knight**, para el movimiento por el mapa y el escalado de las habilidades.



Figura 1. Partido de Nintendo World Cup

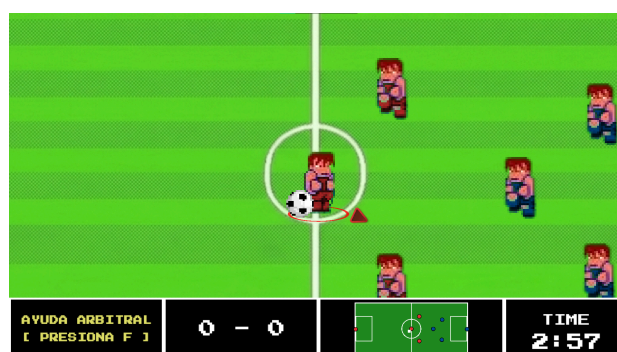


Figura 2. Partido de Ascension

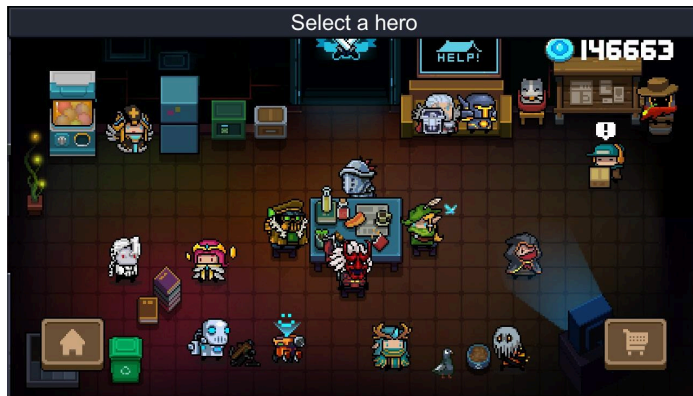


Figura 3. HUB de Soul Knight



Figura 4. HUB de Ascension



## 2. Mecánicas del juego

### 2.1 Sala inicial

En cada liga empiezas desde una **sala inicial**, donde podrás comprar mejoras en la tienda gastando oro o entrenar a tu equipo en el gimnasio usando puntos de esfuerzo (PE). Más adelante se explica cómo se obtienen estos. Desde esta sala también se puede acceder a un campo de entrenamiento donde puedes probar libremente controles y jugadas con unos rivales estáticos, además de practicar los penalties.

Una vez se sale por la puerta de esta sala, se entra en la liga y ya no se puede volver a menos que se pierda un partido o se termine la liga. Las salas que puede contener la liga vienen explicadas en el siguiente apartado.



Figura 5. Sala inicial

|                                                                                     |                                                                                     |                                                                                       |
|-------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------|
|  |  |  |
| Tienda                                                                              | Gimnasio                                                                            | NPC que da acceso al campo de entrenamiento                                           |

Tabla 1. Interactuables de la sala inicial (tienda, entrenamiento)

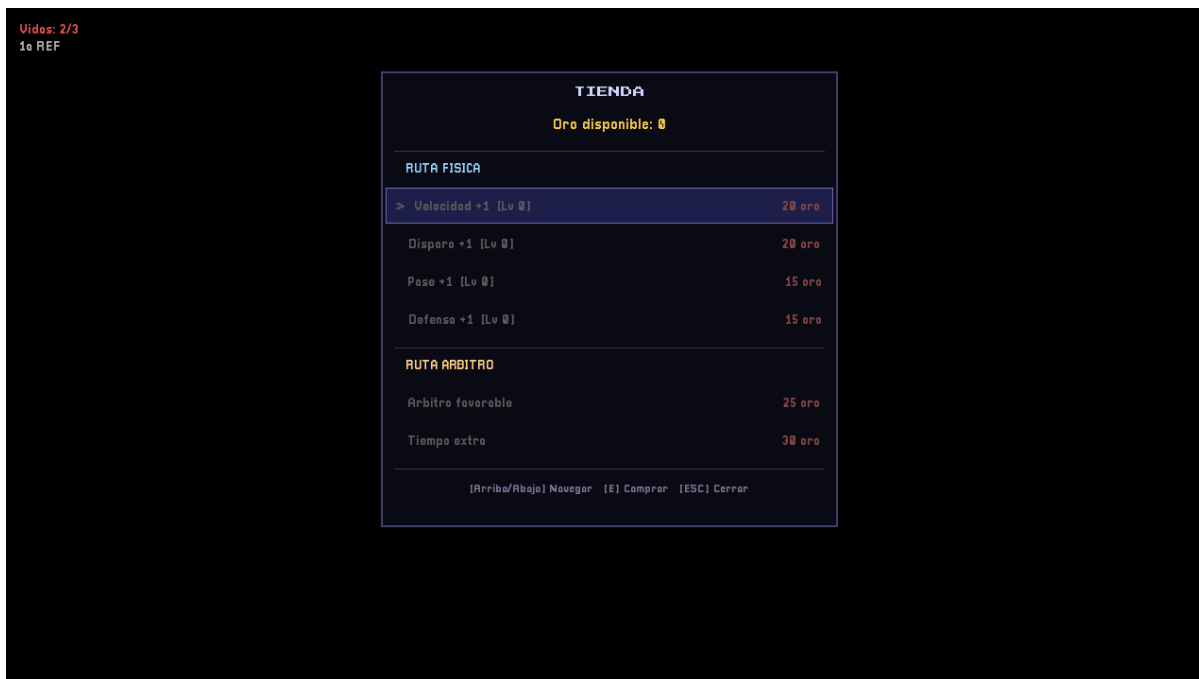


Figura 6. Menú de la tienda



Figura 7. Menú del entrenamiento

## 2.2 Salas del Roguelike

- **Sala de rival:** Sala que da acceso a uno de los partidos de la liga. En ella hay un **rival** que inicia un diálogo introductorio y lleva a la escena donde se jugará el partido.

En cada partido se juega contra un equipo distinto, cada uno con su propio diálogo introductorio y con un color de equipación distinto.

Conforme se avanza de liga, van apareciendo más espectadores en las gradas para dar sensación de que avance en el juego.



Figura 8. Sala de rival en 1ª Federación (primera liga)

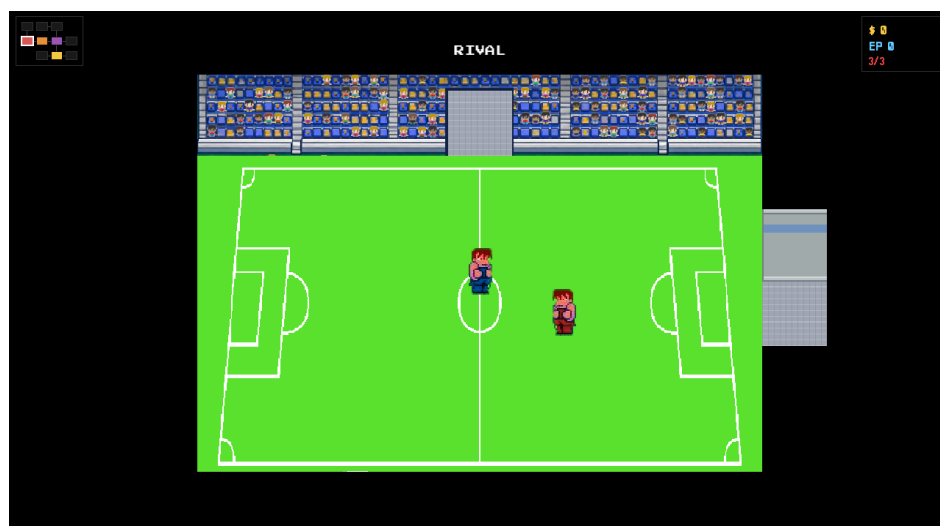


Figura 9. Sala de rival en La Hypermotion (segunda liga)

- **Sala de rival final o boss:** Funciona igual que la sala de rival y **conduce al partido final de la liga.**

Solo se puede acceder a ella tras haber **ganado todos los partidos anteriores** de la liga.

Para diferenciar esta sala de las salas de rival normales de la liga, aparecen considerablemente más espectadores en las gradas. Las salas de boss siempre tienen más espectadores que las de rival normal de cualquier liga, para acompañar el aumento de tensión al llegar a un partido final y la “relajación” al vencerlo y empezar una nueva liga.



Figura 10. Sala de boss de La Liga (última liga)

- **Sala de cofre:** Sala con unas **taquillas** que funcionan como un cofre en que se obtienen recompensas. El jugador tendrá que elegir una de las siguientes:
  - Una cantidad aleatoria de oro
  - Una cantidad aleatoria de puntos de esfuerzo (PE)
  - Un objeto aleatorio que puede ser:
    - Una mejora temporal de una estadística (solo durante la liga actual)
    - Una ayuda arbitral de un solo uso

Cada liga cuenta con **3 salas de cofres**.

Inicialmente se planteó como una sala con un cofre al estilo medieval de los juegos roguelike tradicionales, pero para integrar mejor esta sala con la estética del juego, se decidió convertirla en un vestuario con una taquilla de la que puedes “robar objetos a tus rivales”.



Figura 11. Sala de cofre



Figura 12. Recompensas de un cofre

- **Sala de apuestas:** Sala con una máquina en la que puedes hacer apuestas para el siguiente partido que se juegue. Ofrece tres apuestas de las cuales se puede elegir una:

- Ganar el partido (apostando 10 de oro)
- Ganar el partido con dos goles de diferencia (apostando 20 de oro)
- Ganar el partido con tres goles de diferencia (apostando 30 de oro)

Si en el siguiente partido se cumple la condición apostada, se recupera la apuesta con un 30% extra. Si no, se pierde el dinero apostado.

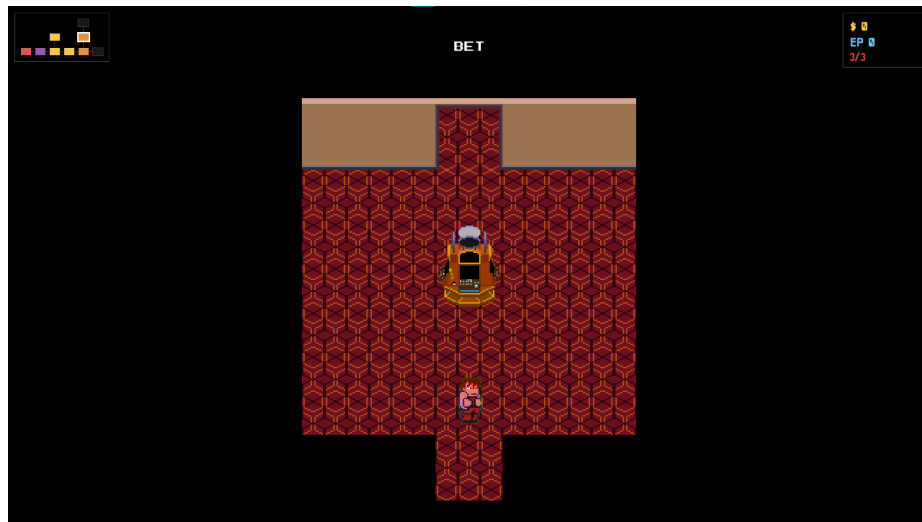


Figura 13. Sala de apuestas



Figura 14. Opciones de apuestas

Esta sala, además, tiene un **evento especial** para dar más vivacidad a la parte roguelike del juego. Se trata de un evento en el que, al entrar a la sala, en ella hay cuatro ultras rivales que muestran un diálogo y luego le dan una paliza al jugador y le roban dinero. Durante el diálogo, se dan tres opciones al jugador que determinarán el porcentaje de oro que se le quitará:



| Acción del jugador | Porcentaje de oro robado |
|--------------------|--------------------------|
| Negociar           | 20%                      |
| Plantar cara       | 33%                      |
| Soltar una gracia  | 50%                      |

Tabla 2. Probabilidades ultras

Independientemente de la opción que se elija, siempre se robará un **máximo de 20 de oro**, para no hacer el juego demasiado injusto.

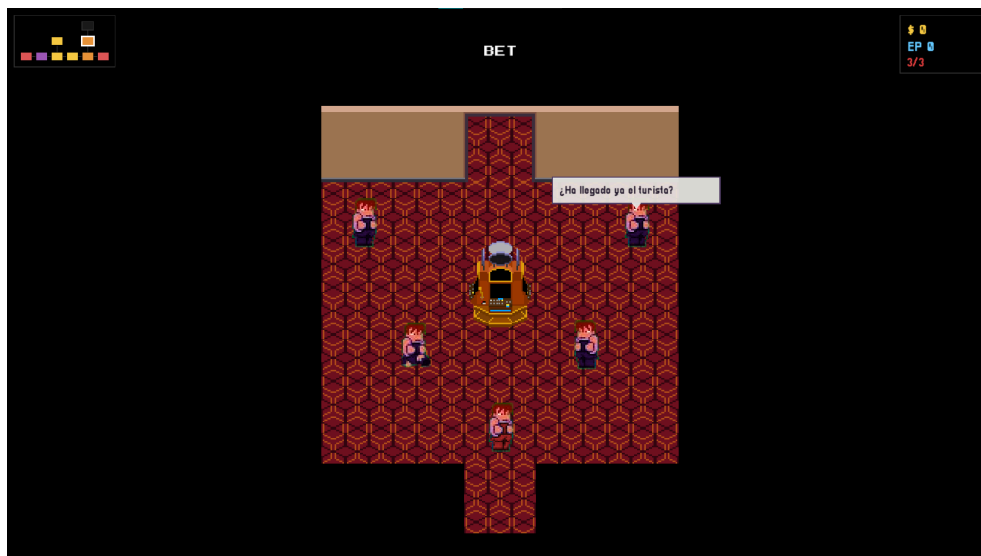


Figura 15. Evento de los ultras

Este evento puede ocurrir un máximo de una vez en cada liga, y lo hace con una probabilidad que se aplica cada vez que se entra a la sala y que va disminuyendo conforme se avanza de liga, para evitar que ocurra con demasiada frecuencia y resulte repetitivo.

| Liga           | Probabilidad de aparición |
|----------------|---------------------------|
| 1ª Federación  | 80%                       |
| La Hypermotion | 30%                       |
| La Liga        | 5%                        |

Tabla 3. Probabilidad de aparición del evento de ultras

- **Sala de amaños:** Sala con un NPC que ofrece **amaños para los partidos**. Estos consisten en **obtener ayudas arbitrales gratuitas** con una duración mayor que las de un solo uso. Pueden ser usadas una vez por partido durante el tiempo establecido, pero conllevan un **riesgo** de que la FIFA te descubra y te haga descender de liga. Esto consiste en una probabilidad que se aplica al final de cada partido mientras dura el amaño (independientemente de si has usado las ayudas) y, si se cumple, se vuelve automáticamente a la liga anterior, aunque con un nuevo mapa para que no resulte tan frustrante retroceder en el progreso del juego. También se conservan las estadísticas obtenidas en la tienda y el campo de entrenamiento, pero se pierden todas las ayudas arbitrales, aunque no se hayan conseguido mediante amaños.

| Amaño            | Duración                       | Ayudas arbitrales recibidas        | Riesgo |
|------------------|--------------------------------|------------------------------------|--------|
| Amaño permanente | Hasta completar la liga actual | Sin faltas en tu contra            | 5%     |
| Amaño fuerte     | Solo para el siguiente partido | Penalti gratuito y ganar empatando | 20%    |

Tabla 4. Especificación de amaños



Figura 16. Sala de amaños



Figura 17. Amaños disponibles

- **Sala de penaltis:** Sala en la que el jugador puede probar las **tandas de penaltis**, que se juegan en **3D**, contra la CPU. Las tandas siguen las reglas del fútbol real: alternancia de turnos de lanzar o parar, gana el equipo con más goles después de 5 lanzamientos por equipo, muerte súbita en caso de empate en esos 5 primeros lanzamientos (en una ronda, si uno marca y el otro falla, gana el que marcó). Ganar estas tandas no es necesario para ascender de liga, pero completarlas de manera exitosa otorga puntos de esfuerzo.

La sala lleva a la escena 3D, la cual tiene un estadio de fútbol, una portería, un portero, un lanzador, un balón y un vacío azul que simula el cielo. Los colores de las equipaciones corresponden al elegido por el jugador (color de su equipo) y al blanco (color por defecto para la CPU en esta sala, en caso de que el jugador haya elegido el blanco, se asigna negro a la CPU).



Figura 18. Sala de penalties



Figura 19. Escena 3D de penalties

## 2.3 Partido

Las salas de rival y de jefe conducen a una nueva escena donde se juegan los partidos. Estos transcurren en un campo horizontal donde se enfrenta el equipo del jugador contra el del rival con el que se habló anteriormente.

Los equipos son de **tres integrantes más el portero**. El juego está diseñado para permitir añadir el número de jugadores que quieras, pero por decisión de diseño, en favor de mejorar la jugabilidad, se decidió escoger este número de jugadores fijo.

El jugador controla a uno de los integrantes de su equipo, mientras el resto, son controlados por la **IA**, que viene detallada más adelante.

Mientras no se tiene la posesión del balón, el jugador puede ir cambiando el control entre los distintos integrantes del equipo, siempre siendo el más cercano al balón el que pasa a ser controlado, a excepción del portero.

Cuando se tenga el balón, el jugador siempre controlará al integrante que lo tenga.



Figura 20. Presentación del partido

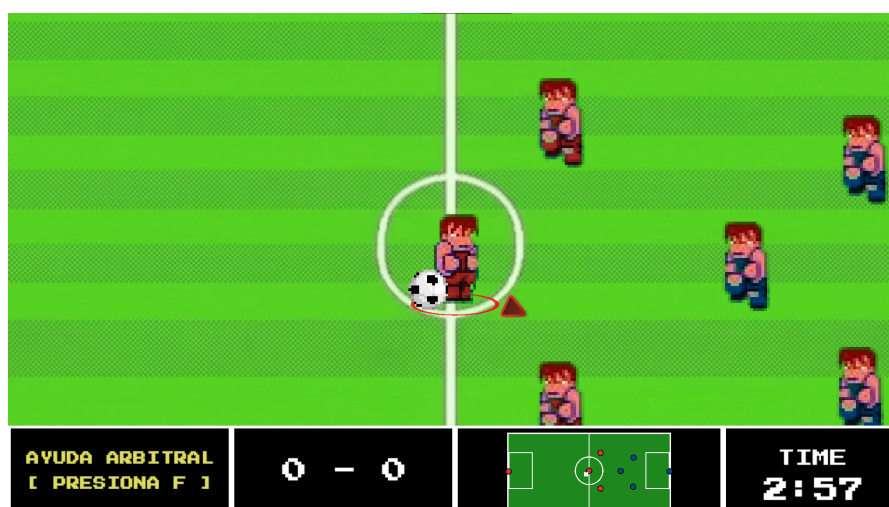


Figura 21. Partido

## Acciones disponibles

- Sin posesión del balón
  - Hacer segadas para quitarle el balón al rival
  - Cambiar de jugador
- Con posesión del balón
  - Pasar el balón a un compañero (y pasar a controlar a este)
  - Lanzar a portería

Los pases y los tiros funcionan mediante conos de precisión. Con una baja estadística de pase y tiro el cono será amplio por lo cual la precisión es peor. Si tienes subidas ambas estadísticas los conos serán muy estrechos por lo que la precisión aumenta.

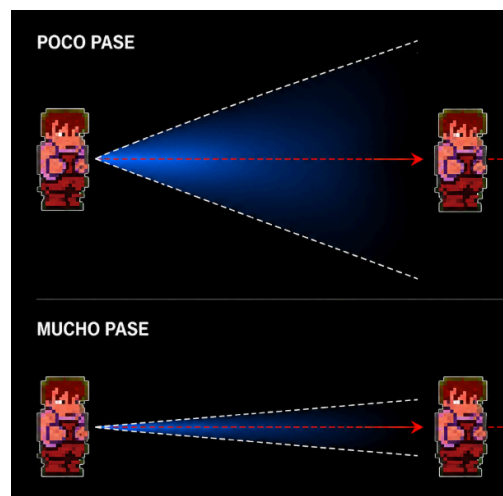


Figura 22. Ejemplo de cono de pase con la dirección introducida en rojo

Hay tres tipos de lanzamientos y pases:

- Normal
- Alto
- Curvo

## Reglas

- La duración de los partidos es de **3 minutos**. Cuando este tiempo termina, la victoria es para el equipo que haya marcado más goles. En caso de empate, el resultado del partido se decide en el minijuego de penaltis 3D.
- **No hay fuera de juego ni fuera de campo**, para mantener partidos rápidos y dinámicos, más propios del estilo roguelike que buscamos. Cuando el balón llega a los límites del campo, rebota.





Figura 23. Campo de juego

- Cuando cualquier integrante del equipo del jugador o del rival hace una segada, hay un **10% de probabilidad de que se le pite falta**. Cuando esto ocurre, los jugadores de ambos equipos se reagrupan y saca el balón el equipo contrario al que ha hecho la falta.

En el caso de perder un partido, **se vuelve a la sala inicial**, donde se pueden mejorar estadísticas con el oro y los PE obtenidos a lo largo de la liga. Después, al ingresar de nuevo a la liga, te encuentras en el mismo mapa de antes con la posibilidad de entrar a jugar contra el mismo rival que te ha ganado antes. Si decides entrar a la sala de rival, los compañeros de equipo te bloquearán las puertas, de manera que no se puede seguir avanzando por el mapa hasta que se gane el partido.



Figura 24. Sala de rival con las salidas bloqueadas

Cada vez que se gana un partido, se obtienen **20 de oro y 20 puntos de esfuerzo**.

En cada liga, se tienen 3 vidas, perdiendo una cada vez que se pierde un partido. Al perder todas las vidas, el jugador descenderá de liga, de la misma manera que cuando es descubierto con un amaño, pero sin perder las ayudas arbitrales.

## Estadísticas

En los partidos también entran en juego las estadísticas que el jugador va mejorando en las salas de la liga. Estas son:

- **Velocidad** → Aumenta la velocidad de movimiento de todos los jugadores del equipo.
- **Tiro** → Aumenta la potencia y precisión de los tiros.
- **Pase** → Aumenta la potencia del pase y reduce el cono de precisión.
- **Defensa** → Aumenta la inteligencia defensiva de la IA aliada.

Se empieza el juego teniendo todas estas estadísticas a 0 y pueden llegar hasta **nivel 10**.

Aparte del equipo del jugador, el equipo rival también tiene estadísticas, que suponen el principal aumento de dificultad a lo largo del juego, ya que aumentan todas un 5% en cada partido y un 10% al avanzar de liga.

Sin embargo, el equipo rival no tiene estadística de defensa, ya que esta determina el comportamiento de la IA, y esta no aumenta de dificultad en el equipo rival.

## Ayudas arbitrales

Además de las estadísticas, las ayudas arbitrales son otra mecánica que el jugador puede ir consiguiendo en las salas de la liga y activar en cualquier momento del partido. Una vez seleccionadas estas se **activan y consumen**, con la excepción del minuto extra que puede ser activado y desactivado a voluntad, aunque se consume de igual manera.

Estas son:

- **Sin faltas en contra**: Nunca se pitan faltas en contra del jugador.
- **Penalti gratuito**: Se da al jugador la oportunidad de tirar un penalti en el minijuego 3D que contaría como un gol más en el partido.
- **Tiempo extra**: Añadir un minuto más al marcador.
- **Ganar empatando**: En caso de empatar, el jugador gana inmediatamente el partido sin necesidad de penaltis.

A diferencia de las estadísticas, el equipo rival solo tiene ayudas arbitrales en el partido final de cada liga, para darles una dificultad extra, siendo estos los puntos de tensión máxima del juego. En cada liga, el equipo rival tiene distintas ayudas arbitrales:

| Liga           | Ayudas arbitrales del rival    |
|----------------|--------------------------------|
| 1ª Federación  | Minuto extra                   |
| La Hypermotion | Minuto extra y ganar empatando |
| La Liga        | Dos penaltis gratuitos         |

Tabla 4. Ayudas arbitrales de los rivales



En el caso de los penaltis gratuitos, el jugador juega al minijuego 3D en el rol de defender, pudiendo evitar que el rival marque esos goles.

## Decisiones de diseño

Se decidió que los partidos se jugarán en una escena distinta de la sala del rival, ya que se necesitaba un campo más grande en que todos los jugadores pudieran moverse sin problema, mientras que la parte roguelike exigía una sala más pequeña, que se pudiera atravesar rápidamente para moverse por el mapa con facilidad.

Además, en la parte roguelike se eliminaron las porterías, que podían **obstaculizar demasiado el movimiento** al entrar a la sala desde los laterales y en los partidos se incluyeron vallas publicitarias para explicar los rebotes del balón al no haber fueras. Sin embargo, se ha usado la misma imagen original del campo para darles la máxima coherencia posible.

El tamaño del campo está ajustado para que **siempre se vea al menos una de las líneas verticales blancas en la pantalla**, de manera que el jugador siempre sepa dónde está. Sin embargo, como la cámara siempre sigue al balón, es posible que el personaje controlado por el jugador u otros integrantes a los que se les quiera pasar el balón queden fuera de la pantalla. Por esta razón, posteriormente, se decidió añadir un minimapa para poder encontrar a los jugadores fácilmente.

Se prefirió que la cámara siguiera al balón en lugar de al jugador, ya que es más fácil perder el balón de vista al moverse rápidamente y no ser siempre controlado por el jugador, además de que su posición es lo más importante para el partido, ya que determina la victoria o la derrota.

## Controles

| CONTROLES           |                        |                    |
|---------------------|------------------------|--------------------|
| ACCION              | PC                     | MANDO              |
| Movimiento          | WASD                   | Joystick izquierdo |
| Apuntar / Direccion | Raton                  | Joystick izquierdo |
| Cambiar de jugador  | Tab                    | L1                 |
| Pase                | Clic derecho           | Cruz               |
| Pase alto           | Shift + Clic derecho   | Triangulo          |
| Pase curvo          | Ctrl + Clic derecho    | R1 + Cruz          |
| Tiro                | Clic izquierdo         | Circulo            |
| Tiro alto           | Shift + Clic izquierdo | L1 + Circulo       |
| Tiro curvo          | Ctrl + Clic izquierdo  | R1 + Circulo       |
| Pausa               | Escape                 | START              |
| Segada              | E                      | Cuadrado           |
| Teclos de direccion | Flechos                | Cruceta            |
| Ayudas arbitrales   | F                      | L2                 |

[Click] o [Esc] para Volver

Figura 25. Controles juego

Los controles de los penaltis están explicados en la escena 3D, en un panel justo debajo del marcador (que también indica el turno de disparo/parada): disparar y lanzarse para parar con el **click izquierdo**, intercambiar la vista a primera o tercera persona (solo al disparar) con la **tecla F** y rotar la cámara para ver el escenario 3D manteniendo pulsada la **rueda del ratón** y moviéndolo (tanto en tercera como en primera persona).

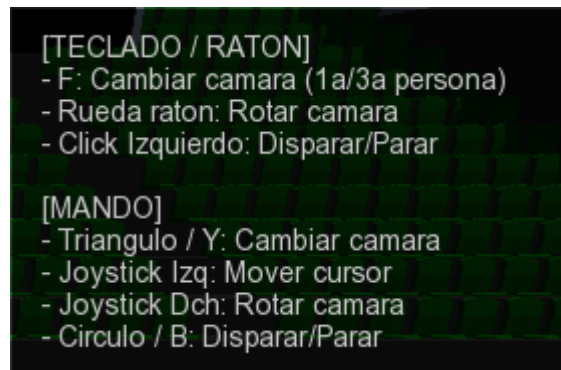


Figura 26. Controles penalties

Atajos: Se han dejado implementados una serie de atajos que se pueden activar dentro de la liga para poder recorrer el juego completo sin necesidad de completar todo:

- F1: Perder un partido (Te quita una vida y te devuelve a la sala inicial)
- F2: Ascender de liga
- F3: Desbloquear la sala de rival final (En 1a REF sirve para probar un partido con la IA más compleja)
- F4: Recibir 100 de oro y 100 de PE

## 2.4 Tutoriales

Se ha implementado un **sistema de diálogo con un manager**. El manager es el encargado de ser tu **guía en el juego**, comentando el objetivo del juego, que función tiene cada sala del mapa o explicando en partido alguna situación concreta. Además, cumple el rol de tu entrenador, animándote, presionando si pierdes y celebrando tus victorias.



Figura 27. Diálogo del manager

También, se ha implementado en la sala inicial un interactuable, (la persona con la camiseta blanquiazul) que te permite ir a un **campo de entrenamiento** para poder practicar y acostumbrarse a los controles contra un rival inmóvil que tan solo puede coger el balón y un portero. También te permite practicar a tirar penaltis sin recibir recompensa.

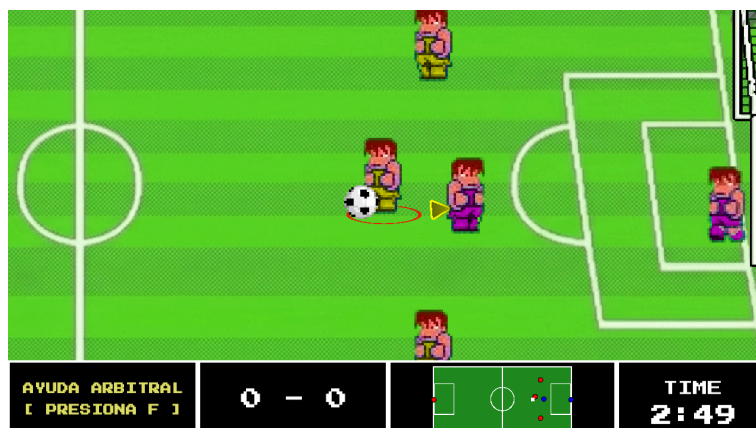


Figura 28. Campo de entrenamiento

### 3. Gráficos

Debido a nuestras escasas cualidades artísticas, los sprites y fondos de la parte 2D han sido obtenidos de Internet o generados mediante IA.

Para la parte roguelike, queríamos inspirar las salas en el interior de un estadio de fútbol (pasillos, vestuarios...), pero al ser una temática bastante original para un juego roguelike, fue difícil encontrar recursos específicos más allá del campo de juego. Así que utilizamos imágenes de cosas más variadas y **las adaptamos a nuestro juego**.

#### 3.1 Salas de la liga

Para que todas las salas tuvieran coherencia pese a que los fondos venían de fuentes distintas, recortamos pequeñas piezas de las paredes y los suelos con la herramienta Paint para **reconstruir las salas con la forma y tamaño que nosotros queríamos**.

Las salas se componen de una imagen del suelo y otra de la pared principales, así como de las cuatro puertas de acceso de la sala: arriba, abajo, izquierda y derecha. Además, también hay dos imágenes de las paredes de las puertas laterales. Al entrar en una nueva sala, la clase DoorSystem se encarga de **determinar qué puertas hay que colocar** según en qué posiciones haya salas adyacentes en el mapa.

A continuación se muestran, a modo de ejemplo, los assets de la sala de amaños:

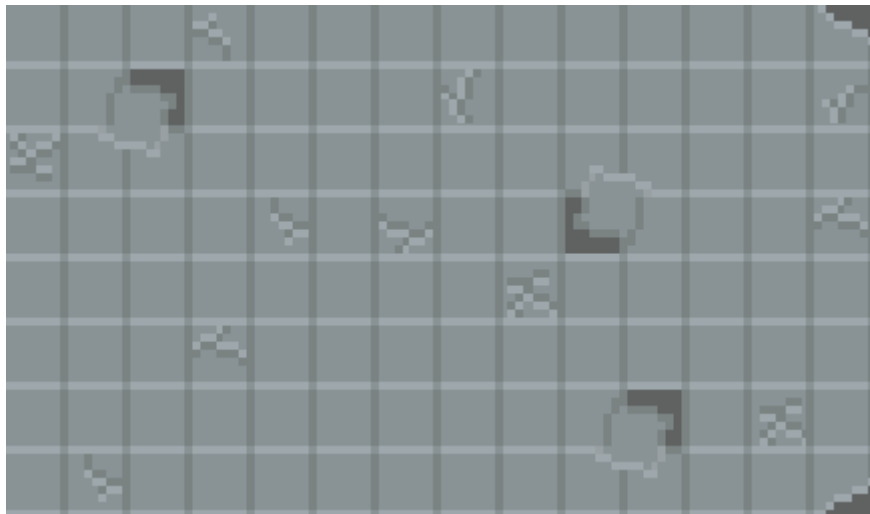


Figura 29. Suelo principal de la sala de amaños



Figura 30. Pared principal de la sala de amaños

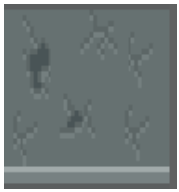
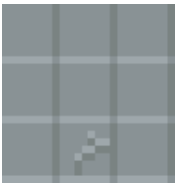
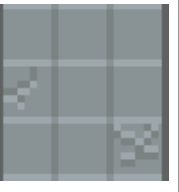
|                                                                                   |                                                                                   |                                                                                   |                                                                                   |                                                                                     |                                                                                     |
|-----------------------------------------------------------------------------------|-----------------------------------------------------------------------------------|-----------------------------------------------------------------------------------|-----------------------------------------------------------------------------------|-------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------|
|  |  |  |  |  |  |
| Suelo puerta izquierda                                                            | Pared puerta izquierda                                                            | Suelo puerta derecha                                                              | Pared puerta derecha                                                              | Suelo puerta abajo                                                                  | Suelo puerta arriba                                                                 |

Tabla 5. Celdas de suelo y pared



Figura 31. Ejemplo de montaje de sala de amaños



Figura 32. Ejemplo de montaje de sala de amaños

- Los fondos de la **sala de amaños** se obtuvieron de un tileset de una fábrica, que tenía tuberías y paredes y suelos agrietados, y se usaron para crear una sala que fuera como un cuarto de máquinas o de almacenaje, un lugar discreto y poco frecuentado del estadio que presentará los amaños como algo clandestino e ilegal. Para el hombre que ofrece los amaños, se buscó un sprite de gángster o mafioso por la misma razón.

- La **sala de apuestas** está inspirada en un casino, y todos sus assets provienen del tileset de un casino. Estos fueron los más fáciles de encontrar, ya que es más fácil encontrar casinos y salas de apuestas en juegos de estilo RPG retro.
- Para la **sala de cofres**, inspirada en un vestuario como se cuenta en la sección anterior, se obtuvieron las paredes y los suelos del tileset de un baño convencional, y las taquillas de un colegio.
- Para la **sala de partidos**, el campo fue lo más fácil de encontrar, ya que encontramos una página web con todos los assets del **Nintendo World Cup**, el juego en el que se inspiran los partidos del juego. Se usó el mismo campo, aunque no tenía paredes, ya que es un espacio al aire libre. Sin embargo, para una sala del roguelike, se necesitaba tener algo que hiciera de fondo para que tuviera la misma estructura que el resto. Se pensó que unas gradas era la opción más lógica, pero no pudimos encontrar nada en Internet que encajara, así que se generó una imagen con ChatGPT.  
Las paredes y suelos de las puertas, que simulan los pasillos del estadio, se obtuvieron de un **tileset de interiores** de un juego de Pokémon.
- Para la **sala de penaltis**, se quiso hacer otro campo de fútbol, pero que se viera más como un potrero, indicando que los penaltis tienen menos relevancia que los partidos para el avance del juego. Se usaron las mismas gradas que en los partidos (aunque más pequeñas) por la dificultad para encontrar algo en su lugar, pero para el suelo se optó por hacer un terreno de hierba sin líneas, con hierbajos y tierra por los bordes, para que pareciera un campo más improvisado. Se utilizó un **tileset de campo** y se recortó para darle la forma final. Los hierbajos se añadieron a mano, distribuyéndolos aleatoriamente por la imagen.
- La sala inicial o hub, al ser un lugar donde se pueden mejorar las habilidades del equipo, se decidió diseñarlo como un gimnasio. Se mejoran estadísticas en cintas de correr y se compran objetos en máquinas expendedoras. Más adelante, surgió la idea del campo de entrenamiento y, como no se podía directamente meter el campo dentro del gimnasio, se representó como un NPC que pretende ser uno de los rivales estáticos que aparecen en el entrenamiento. La pared y el suelo se obtuvieron también de un tileset de interiores de Pokémon, ya que tenía muchas cosas genéricas y la decoración de los gimnasios suele ser neutra y austera.

## 3.2 Partidos

Esta fue la parte la más fácil de hacer, ya que pudimos encontrar todos los assets del juego que sirvió de inspiración para esta parte, el **Nintendo World Cup**. Los jugadores y el campo provienen de aquí.

Las vallas publicitarias se añadieron posteriormente, como se explica en el apartado anterior, y se generaron utilizando IA generativa.

### Animación de Sprites

Se tuvo que diseñar una clase llamada **Animation** para gestionar todas las animaciones de los sprites, que **consiste en cargar en cada frame las distintas texturas para esa acción de ese sprite**, junto con la lógica de fotogramas por segundo, la aplicación de **máscaras** de color y con opcionalidad a reproducción en bucle o no. Los frames de cada animación están

guardados en carpetas, no se usan spritesheets si no que las dejamos ya cortadas para aliviar la carga al ordenador.

La clase utiliza **vectores** para almacenar tanto la **textura del sprite como** la de las **máscaras**. Carga todos los frames de las carpetas indicadas y en lugar de vincular las animaciones a la tasa de refresco del hardware, lo que provoca una aceleración según la CPU, el método update recibe el tiempo diferencial de la CPU, y es el encargado de pasar de fotograma cuando haya pasado un tiempo. Ofrece soporte nativo para animaciones en bucle o de finalización única.

Un desafío en el diseño fue permitir la personalización de las equipaciones de los diferentes equipos rivales utilizando las mismas animaciones base. Al principio se consideró utilizar shaders, pero no dió el resultado esperable. También se podía haber modificado cada sprite por cada color pero habría sido un malgasto de recursos, tiempo, y sin poder ser modulable. Por tanto, lo que se optó fue un **sistema de máscaras de color** acoplado al ciclo de la clase, donde se modificaron los sprites originales **separando las camisetas del propio jugador**, dejando los frames sin sus troncos y extremidades, para hacer una mascara con ellos en escala de grises, y con la función de setColor de SFML, añadirle el color deseado. Así que se dibuja al jugador por un lado y las camisetas a la vez.



Figura 33. Textura del sprite y máscara en escala de grises

También se añadió **animación a la pelota de fútbol**, la cual al principio giraba dando una sensación poco realista. El balón requirió un tratamiento técnico individualizado puesto que no había ningún sprite en internet que encajara con la estética de nuestro proyecto o tuviera una animación adecuada. Es por esto que **se rediseñó** a partir de un modelo, los frames de un balón para que se integrase bien y tuviese un movimiento natural y orgánico. Estas modificaciones y las anteriores mencionadas se realizaron usando la herramienta de edición *gimp*. Para la **simulación de la altura**, además de añadir un movimiento sinusoidal para captar el efecto que se produce en el aire, se **pinta una sombra** que cambia de tamaño según la altura de ésta, consiguiendo así una sensación “3D”.



Figura 34. Textura del balón de fútbol en distintos frames junto con su sombra de distinto tamaño según la altura



## 4. Inteligencia Artificial

La IA se usa para programar el comportamiento del equipo rival y tus compañeros de equipo durante los partidos. Decide sus acciones mediante una máquina de estados los cuales son: defender, atacar, perseguir balón, pasar, disparar y volver a posición. Además realiza un posicionamiento por zonas.

### 4.1 Escalado

La IA escala de dos formas:

- Los dos primeros partidos la IA defensiva **no te presiona**, es decir, solo te busca quitar el balón si entras en su rango, pero no va a por ti. De esta forma se suaviza la curva de aprendizaje al inicio del juego. A partir del primer partido de ascenso (primera sala de rival final), siempre te va a salir un defensa a presionarte para provocar que tengas que jugar más con los pases y dificultar el ataque.
- Con cada partido jugado la IA **escala sus estadísticas** mediante multiplicadores, además se produce otro salto si asciendes de una liga a otra. Tus estadísticas escaladas al máximo son mayores que las de la IA escaladas al máximo para asegurar que sea viable jugar contra el último rival de la IA. Cálculos para el escalado de estadísticas:
  - Multiplicador de liga =  $n^{\circ}$  de liga \* 0.1
  - Multiplicador de partido =  $n^{\circ}$  de partido \* 0.05
  - Multiplicador final = Multiplicador de liga \* Multiplicador de partido

### 4.2 Comportamiento

La máquina de estados se compone de:

- **Defender:** Accede a este estado cuando un jugador rival **tiene posesión de balón** y el jugador ya está posicionado **en su zona defensiva**. Con el primer escalado completo, la IA defensiva funciona con presión por zonas. Si detecta que estás en su zona va a ir a presionarte para robarte el balón. De esta forma se regula que tan solo un rival te salga a presionar y no sea tan agobiante.

Si entras en el rango de acción del rival va a intentar realizar una entrada para quitarte el balón.

Si acabas de realizar una acción, como un pase a un compañero, la IA rival tiene un **tiempo de espera** para no realizar una entrada instantánea al jugador que ha recibido el pase, alentando a los usuarios a usar pases.

En la siguiente figura se muestran las zonas en caso de defensa del equipo izquierdo donde el color rojo corresponde al defensa superior, el azul al defensa inferior y el morado al delantero.



Figura 35. Zonas defensivas (atacan el lado derecho)

- **Atacar:** Accede a este estado cuando un jugador aliado tiene **posesión del balón**. Cada jugador tiene una zona de ataque por la cual se mueven. Si te acercas conduciendo a un jugador de tu equipo se va a mover para intentar habilitar que le hagas un pase.

También si le cortas el paso hacia la portería, **evalúa cual es el mejor compañero para realizar un pase** y lo hace. De la misma manera, si está lo suficientemente cerca de la portería para realizar un tiro pero ve que le están tapando, buscará la opción de realizar un pase a un compañero. Si se encuentra a una distancia adecuada cambia al estado de “**disparar**”.

En la siguiente figura se muestran las zonas en caso de ataque del equipo izquierdo donde el color rojo corresponde al atacante superior, el azul al atacante inferior y el morado al delantero.



Figura 36. Zonas ofensivas (atacan el lado derecho)

- **Perseguir balón:** Cada vez que hay un balón libre, **el jugador más cercano al balón** (calculado mediante distancia euclídea) va a por él para quedárselo.

- **Pasar:** Si se cumplen las condiciones explicadas anteriormente para realizar un pase, se busca el mejor compañero calculado mediante la **distancia euclídea**, como está **posicionado en el campo** y cuánta **presión rival** tiene encima.
- **Disparar:** Si se encuentra a una **distancia adecuada** de la portería y no tiene **nadie delante** que tape el tiro. Realiza aleatoriamente uno de los tipos de tiro disponibles (raso, alto o curvo) con la misma probabilidad cada uno.
- **Volver a posición:** Se accede a este estado cuando un jugador rival tiene **posesión de balón** y el jugador **no** está posicionado en su zona defensiva. Cada jugador tiene dos puntos de referencia distintos: Para un saque de puerta y para su posición defensiva. Ambos se calculan con una **distancia euclídea** lo que provoca la creación de las zonas de movimiento.

Durante este estado, también intenta quitar el balón a un rival si este entra en su radio de acción. En el momento que el jugador ha vuelto a su posición, es decir, está dentro de su zona, pasa al estado de “**defender**”.

### 4.3 Portero

La IA del portero se regula mediante una **máquina de estados** los cuales son: posicionar, salida, parar, parando, sacar.

- **Posicionar:** Se posiciona en la portería siguiendo el balón en el eje Y. De esta forma siempre se encuentra en una buena posición para parar el balón.
- **Salida:** El portero **sale a presionarte si te acercas con una probabilidad del 20%**. Es una forma de no acostumbrar al usuario a que siempre realice las mismas formas de tiro ya que hay ocasiones que el portero va a salir a por ti. Si ve que te alejas, vuelve a su posición.
- **Parar:** Entra en este estado cuando un jugador del equipo rival realiza un tiro. Si el tiro es viable, es decir, llega al portero, mediante una campana de Gauss se calcula la probabilidad de parar el balón. Si es parada pasa al estado de **parando**. Si es gol, se realiza el saque de centro y va al estado de **posicionamiento**.

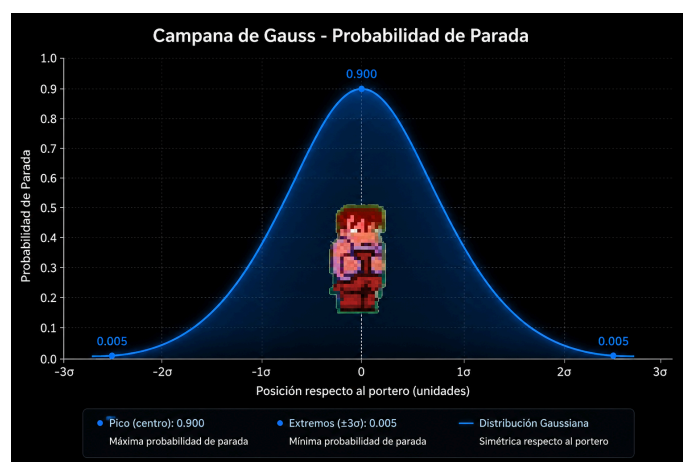


Figura 37. Campana de Gauss del portero

- **Parando:** Si se ha cumplido la probabilidad de parada, se comprueba continuamente si el balón ya ha llegado a la altura del portero para que la animación sea lo más fluida posible y se realiza la parada.
- **Sacar:** Si el portero tiene el balón, espera unos segundos y realiza un saque alto al medio del campo, después vuelve al estado de **posicionamiento**.

#### 4.4 Ayudas arbitrales

Esta mecánica es exclusiva para el partido de ascenso de cada liga. Cada uno de los 3 rivales cuenta con una o varias de estas ventajas para hacerlo más interesante.

| Liga           | Ayudas arbitrales del rival    |
|----------------|--------------------------------|
| 1ª Federación  | Minuto extra                   |
| La Hypermotion | Minuto extra y ganar empatando |
| La Liga        | Dos penaltis gratuitos         |

Tabla 6. Ayudas arbitrales de los jefes

Tanto el rival de 1ª Federación como el de la Hypermotion activan todas sus ayudas nada más comenzar el partido. Sin embargo, el rival de La Liga activa uno de sus penalties cuando el tiempo restante sea menor a 2 minutos y el segundo lo activará en caso de ir perdiendo y que queden menos de 40 segundos en el reloj.

Todo esto te lo notifica el manager al enfrentarte a cada uno de los rivales por primera vez.

## 5. Generación del mapa

El mapa de cada liga se genera **aleatoriamente**, para contribuir a que el jugador pueda echar muchas partidas al juego sin que se sienta todo el rato lo mismo. Además, así, cada liga también se siente **distinta de la anterior** aunque las salas y las mecánicas no cambien demasiado.

Para generar el mapa, se utiliza un **algoritmo procedural** que funciona con una frontera y una bolsa de tipos de salas. La bolsa es un vector que contiene todas las salas que tiene que haber en el mapa según el número de liga, con elementos repetidos para salas que aparecen más de una vez. Las salas de boss, por restricciones que se indican posteriormente, **no se incluyen en esta bolsa**.

|                | Salas de rival | Salas de cofre | Salas de penaltis | Salas de amaños | Salas de apuestas |
|----------------|----------------|----------------|-------------------|-----------------|-------------------|
| 1ª Federación  | 2              | 3              | 1                 | 1               | 1                 |
| La Hypermotion | 3              | 3              | 1                 | 1               | 1                 |
| La Liga        | 3              | 3              | 2                 | 1               | 1                 |

Tabla 7. Número de salas de cada tipo por liga (+ una sala de boss en cada liga)

Para generar la primera sala del mapa, se elige **un elemento aleatorio de la bolsa** y se elimina de esta. Este elemento indica el tipo de sala que se generará, que será en la que aparezca el jugador al entrar a la liga. Esta sala también es la primera que se añade a la frontera.

La frontera es otro vector de salas, que en este caso guarda las salas que ya se han generado y sus conexiones libres. Se entiende por conexión libre **una de las cuatro direcciones** (norte, sur, este y oeste) donde puede haber una puerta de acceso a otra sala, pero aún no la hay.

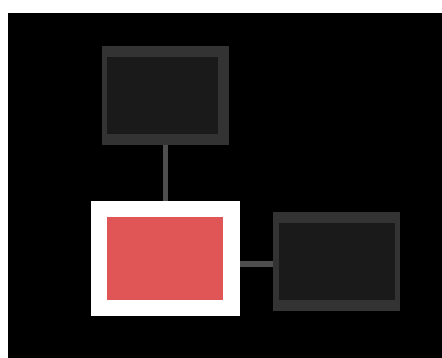


Figura 38. Sala con dos conexiones en uso (norte y este) y dos libres (sur y oeste)

Para generar las siguientes salas, su posición se determina eligiendo **una sala aleatoria de la frontera**. Después, se elige una conexión libre aleatoria que será por donde se acceda a la nueva sala, de manera que nunca queden salas inconexas. El tipo de sala se obtiene de nuevo de la bolsa. Por último, la nueva sala se añade en la frontera y se comprueba si la

sala desde la que se accede **ya tiene sus cuatro conexiones** en uso, en cuyo caso se elimina de la frontera para no salir elegida otra vez. Este proceso se repite hasta que la bolsa de tipos de salas quede **vacía**.

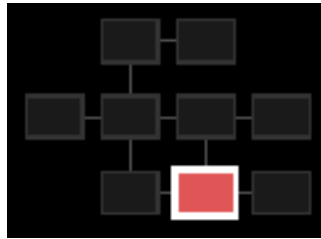


Figura 39. Ejemplo de mapa generado

## 5.1 Alternativa de diseño

Otra opción que se consideró fue guardar en la frontera directamente **todas las conexiones libres**, independientemente de la sala de la que partieran. Esto generaría mapas con formas más homogéneas (cuadradas o rectangulares), ya que el algoritmo tendería a rellenar los huecos donde hubiera varias conexiones orientadas hacia el mismo lugar. Por otro lado, con la implementación elegida, se obtienen formas **más variadas y ramificadas**, ya que todas las salas tienen la misma probabilidad de ser expandidas, haciendo que se puedan generar las siguientes en cualquier lugar con la misma probabilidad. Se prefirió esta opción por ser más típica en juegos roguelike, ya que da más sensación de exploración, **simulando un laberinto**. Además, esto hace que visualmente se note más cómo cada mapa es diferente.

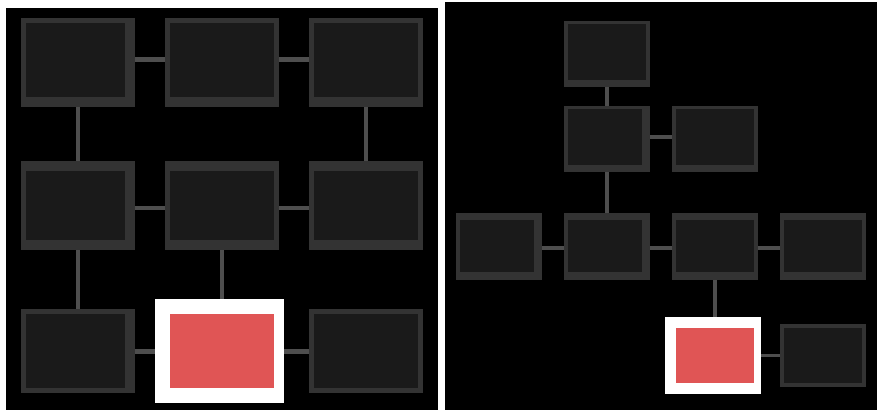


Figura 40. Mapa homogéneo vs. mapa ramificado

## 5.2 Superposición de salas

Cuando se genera una sala, se crea una conexión con la sala desde la que se ha generado, pero también hay que comprobar si la nueva sala tiene otras adyacentes en las demás direcciones que se hayan generado anteriormente. Si no, estas direcciones están técnicamente **libres**, y se podrían generar salas en ellas que se superpusieran a las ya existentes.



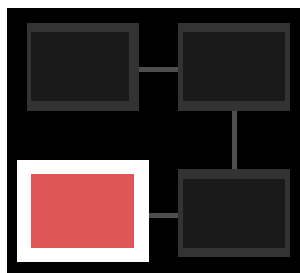


Figura 41. Sala con una adyacente desconocida al norte

Para evitar esto, se añadió una **matriz de coordenadas** en la que se van guardando las posiciones de todas las salas que se van creando, partiendo desde la primera que se coloca en (0, 0). Así, al generar una nueva sala, se puede saber si tiene otras adyacentes. A partir de aquí, se presentan dos opciones de diseño: marcar las conexiones como “**en uso**” para evitar superposición, pero sin crear realmente conexiones con las salas adyacentes, o **crearlas permitiendo bucles**.

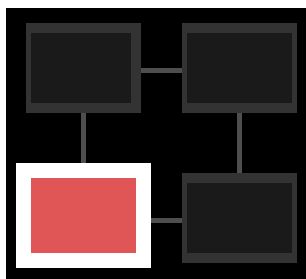


Figura 42. Bucle en el mapa

Para dotar a los mapas de más variabilidad, se optó por una solución intermedia: **a veces se generan bucles y a veces no**.

Cuando se crea una nueva sala, no se comprueba si hay otras adyacentes, sino que se dejan las tres conexiones restantes libres. Después, si más adelante el algoritmo intenta expandir una de esas conexiones, es entonces cuando se comprueba solo si en esa dirección hay una sala adyacente. En ese caso, en lugar de crear una nueva, se crea la **conexión con la ya existente**, dando lugar a un bucle. Sin embargo, las conexiones que el algoritmo nunca llegue a intentar expandir, pueden tener salas adyacentes a las que no se pueda acceder desde allí.

### 5.3 Posición teórica de la sala inicial

Se accede a la liga a través una puerta de la sala inicial que está orientada hacia el norte. Por lo tanto, aunque no haya una puerta para volver a esta desde la liga, se entiende que está ubicada al sur de esta. Por esa razón, no tendría sentido que desde la primera sala de la liga hubiera otra sala ubicada al sur, ya que, en el mundo real, se superpondría con la sala inicial. Para evitar esto, así como generaciones más extrañas, como salas rodeando la supuesta posición de la sala inicial, se evita generar salas más abajo de la primera. Esto se consigue fácilmente con la matriz de coordenadas, que permite saber si la sala que se va a generar está en la altura 0, en cuyo caso se marca la conexión sur como “en uso”, aunque no haya realmente ninguna sala conectada.

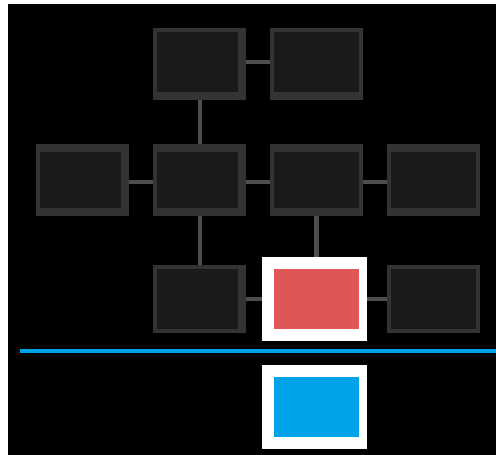


Figura 43. Mapa generado indicando en azul la posición teórica de la sala inicial.  
No se pueden generar salas por debajo de la línea azul.

## 5.4 Sala del boss

Por último, la sala del boss tiene una regla especial: tiene que ser un nodo hoja del mapa, es decir, estar conectada solo a una sala. Esto se debe a que no se puede entrar a esta sala hasta que se completa el resto de partidos de la liga, por lo que si se encuentra en medio del camino hacia otras salas, estas serán inaccesibles hasta que sea desbloqueada. Esto puede ser fatal si una de esas salas que quedan inaccesibles es una sala de rival, de forma que nunca se pueden completar todos los partidos de la liga.

Para conseguir esto, la sala del boss es la última en ser añadida al mapa, y por eso no se añade a la bolsa de tipos de salas. Una vez todo el mapa está generado, el algoritmo vuelve a elegir una sala aleatoria de las que han quedado en la frontera, elige una conexión aleatoria y, esta vez, sí comprueba que no haya salas adyacentes. En caso de no haberlas, se crea ahí la sala del boss. En caso contrario, se elige otra sala de la frontera hasta que esto ocurra.



## 6. Sonido y música

### 6.1 Estilo musical

Para crear un ambiente animado y que incite a la **exploración** se decidió utilizar música obtenida del juego *Pokemon Diamante* en todo lo referente al roguelike.

En el partido se buscó música que fuera **animada** y que ofrezca una cierta **tensión** y **dinamismo**. Optamos por un tema en que simula el sonido de los bombos y tambores de los partidos de fútbol reales. Además se busca que esta música se escuche de fondo y no rompa el ritmo del jugador.

La música elegida para el partido se modificó para poder realizar un bucle **sin ser disruptivo para el oído**.

### 6.2 Uso del sonido

- Inmersión ambiental
  - Público coreando
  - Público abucheando
  - Sonido de victoria al pasarte el juego
  - Sonido al cruzar una puerta
  - Grito cuando ocurre una falta
  - Silbato del árbitro (inicio, faltas, penalti, final)
  - Sonido al golpear un balón en un pase o disparo
- Feedback jugable
  - Confirmar acciones del jugador (compras en tienda, selección de opciones...)
  - Señalar situaciones de riesgo/tensas y eventos importantes (gol, penalti, final de partido)
  - Sonidos distintivos en las salas (interactuar con una apuesta, un amaño...)
  - Sonido al cambiar de opción en el menú

## 7. Aspectos técnicos

### 7.1 Motor

Hemos desarrollado nuestro propio motor sobre la librería de gráficos 2D SFML y C++.

Para la parte 3D del juego (sistema de penaltis), hemos tenido que bajar un nivel de abstracción, usando OpenGL, ya que SFML no da soporte para gráficos 3D, pero está igualmente construida sobre OpenGL.

### 7.2 Lenguaje

El juego se ha programado en C++, ya que es el lenguaje con más compatibilidad que utiliza el motor OpenGL, escrito en C. Usamos este lenguaje también por familiaridad, flexibilidad y tiempos ajustados del proyecto.

### 7.3 Arquitectura

El juego se estructura en **escenas independientes** (menú, hub, liga, partido, penaltis...) gestionadas por un bucle principal que delega eventos, actualización y renderizado a la escena activa.

Cada escena implementa una **interfaz común** (handleEvent, update, render, getNextScene), lo que permite **transiciones limpias** entre estados del juego sin acoplamientos directos.

Los sistemas de juego (economía, progreso del jugador, gestión de la run roguelike, ayudas arbitrales, apuestas...) se implementan como **singletons** o como objetos inyectados en las escenas que los necesitan, separando lógica de presentación.

### 7.4 Guardado

El progreso persiste en un archivo de texto **pares clave valor**. Se guarda el **estado completo**: economía, liga actual, mapa roguelike (tipo, estado y conexiones de cada sala), equipos asignados y flags de tutoriales. El guardado es automático en cada transición relevante o manual cuando el usuario así lo decide.

### 7.5 Configuración y build

El proyecto se construye con **CMake**. Los parámetros configurables del juego (rutas de assets, sprites, sonidos, mapeo de controles) se cargan desde un archivo XML en tiempo de ejecución mediante un ConfigManager centralizado.

## 8. Apartado técnico 3D

El sistema **3D** corresponde a un minijuego de penaltis explicado en el apartado de salas (sala de penaltis). Para esta escena se utiliza la API de OpenGL directamente, ya que SFML no ofrece soporte nativo para gráficos 3D. El objetivo es crear una escena simple pero funcional que represente:

- Portería
- Portero
- Lanzador
- Balón
- Terreno de juego
- Cámara en perspectiva intercambiable a primera persona y rotable

### 8.1 Modelos 3D

Para crear esta escena se han usado **4 modelos 3D**: un estadio de fútbol (incluye terreno de juego y portería), un balón, un jugador lanzador animado y un portero (duplicado para animación, explicado más adelante). Todos ellos son modelos ya creados y se han descargado de forma gratuita de las páginas web <https://sketchfab.com> y <https://poly.pizza>.

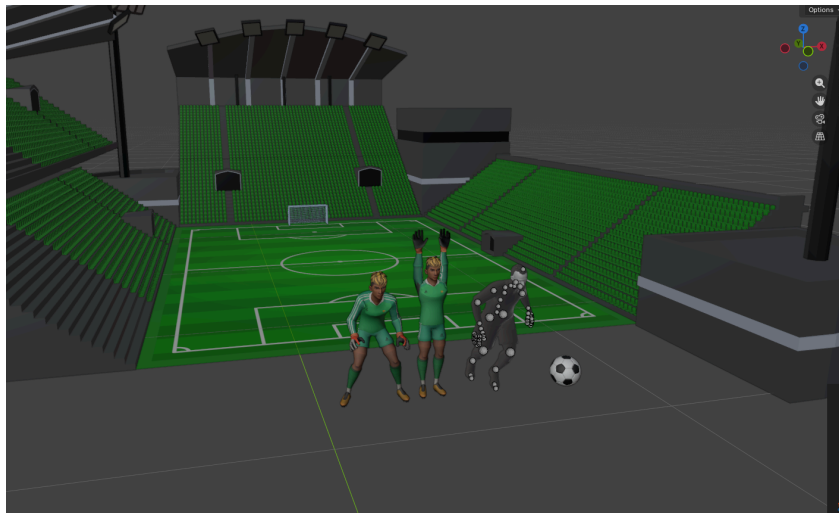


Figura 44. Modelos usados en la escena en Blender

Todos los modelos, a excepción del jugador lanzador, están en **formato OBJ**, que es el más básico y está diseñado para modelos estáticos. El restante está en **formato GLB**, que además permite animaciones.

Para los modelos estáticos se utiliza un pequeño motor 3D para almacenar lo básico de un objeto OBJ y renderizarlo de forma optimizada. Se definen estructuras de datos para representar **vértices y materiales**, que además contienen lo necesario para que un modelo tenga texturas (colores planos RGB o imágenes jpg o png). No se considera que estos modelos tengan huesos, ya que sus animaciones son sencillas y no los requieren. Para leer y cargar estos modelos en memoria se utiliza la librería externa **tinyobjloader**.

Para el modelo animado ha habido que **ampliar considerablemente el motor 3D** descrito, puesto que contiene una animación (no es estático). En este caso se definen estructuras de datos para representar su geometría, materiales, huesos y animaciones. A diferencia de los modelos estáticos, estos modelos sí utilizan **esqueletos formados por huesos** para poder deformar la malla y reproducir animaciones complejas. También se almacenan datos relacionados con las transformaciones, animaciones y jerarquías internas del modelo. Para leer y cargar estos modelos en memoria se utiliza la librería externa *tinyglTF*, compatible con archivos GLB y glTF.

## 8.2 Animaciones

En la escena hay **3 animaciones**: el balón siendo disparado, el portero estirándose para detenerlo y el jugador de campo golpeándolo.

La primera se consigue simplemente modificando la posición del balón (igual que los sprites en 2D). Comienza cuando la animación del jugador de campo llega al frame 18, que es cuando golpea el balón, de este modo se evita desarrollar un sistema de colisiones complejo.



Figura 45. Frame de golpeo del jugador

Para la segunda se utilizan **dos modelos OBJ**: uno del portero preparado para recibir el disparo y otro del mismo portero con el cuerpo estirado, que se sustituye por el primero cuando se dispara el balón y se desplaza igual que el balón. Se hizo así porque no se encontró ninguna animación fluida ya hecha como la del jugador de campo.



Figura 46. Modelos 3D estáticos del portero

Para la del jugador de campo se ha ampliado el motor 3D ya descrito.

### 8.3 Mecánicas de disparo y paradas

Al haber demasiadas opciones de disparo en el área de la portería, se hizo una simplificación. Se dividió la cara frontal de la portería en **6 áreas iguales**, de modo que cuando el jugador elige un punto al que disparar o parar, está eligiendo sin saberlo una de esas áreas. La CPU siempre elige una aleatoriamente. Cuando el jugador dispara, si las áreas elegidas por el jugador y la CPU coinciden, las manos del portero irán al punto elegido por el jugador, sino el portero se lanzará a un punto aleatorio dentro del área elegida. Si el jugador es el portero y las áreas coinciden, el balón se dispara a la posición elegida, sino se dirige a un punto aleatorio dentro del área elegida. Esto hace que la probabilidad de marcar gol sea de cinco sextos y un sexto la de detener el balón.

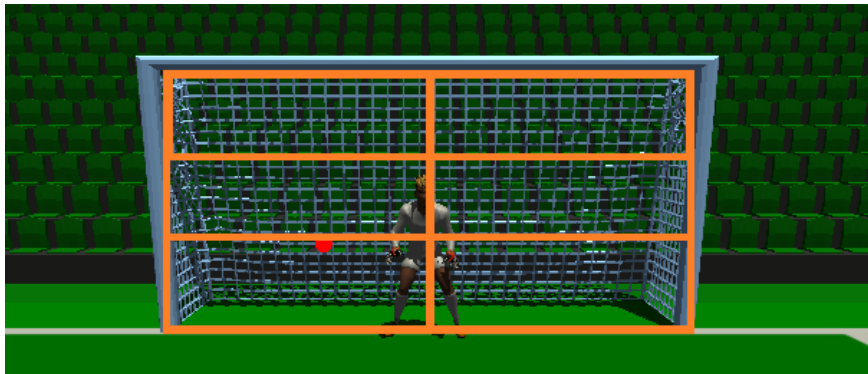


Figura 47. Áreas aproximadas de la cara frontal de la portería

Para detectar a qué área se está apuntando hay un problema, y es que se necesita pasar de **coordenadas 2D de la pantalla a coordenadas 3D** de la escena. Para ello se hace el proceso inverso del pipeline gráfico. Normalmente renderizar es:

$$ScreenCoords = ProjectionMatrix \cdot ViewMatrix \cdot WorldCoords$$

En este caso se pasa de coordenadas de un objeto (en este caso el cursor 2D en plano cercano y lejano) en la pantalla a coordenadas del mundo 3D:

$$WorldCoords = (ProjectionMatrix \cdot ViewMatrix)^{-1} \cdot ScreenCoords$$

Se repite esta fórmula **2 veces**, en la que se sustituyen las coordenadas de la pantalla con las del cursor de apuntado, tanto en el plano cercano con el lejano. Con esto se obtienen 2 puntos reales del mundo 3D con los que se traza un vector. Con la dirección de ese vector se traza un rayo que parte de la cámara. Para saber el área a la que se ha apuntado, se comprueba si el rayo choca con el plano de la portería ya mencionado. De esta manera se obtiene la intersección del rayo visual con el plano 3D de la portería. Comprobando sus coordenadas se sabe si se ha apuntado fuera o dentro de la portería, y si ha sido dentro, su área dentro de la portería.

Para añadir dificultad en el apuntado al disparar, el cursor con el que se apunta se desplaza continuamente de forma aleatoria en torno al cursor real, de modo que se requiere cierta habilidad para apuntar correctamente dentro de la portería y no disparar fuera. No se

consideró apropiado hacer esto cuando el jugador sea el portero, ya que el portero (CPU o jugador) está en mucha desventaja.

## Cámara

La cámara por defecto se coloca en **tercera persona** para visualizar la escena completa. Esta puede cambiar a **primera persona** (visión del modelo de lanzador) cuando el jugador es quien dispara. La cámara se adjunta a uno de los huesos de la cabeza del jugador y se aleja una distancia mínima para no ver el interior del modelo. En ambos casos se puede aplicar offsets a la cámara para ampliar la visión de la escena.

## 8.4 Integración SFML + OpenGL

SFML se utiliza para: Creación y gestión de ventana, Gestión del contexto OpenGL, Input (teclado / ratón / mando), Control del loop principal.

- **Tecnología**
  - OpenGL para renderizado 3D.
  - Integración con SFML para gestión de ventana y contexto.
  - C++ como lenguaje.
  - Uso de matrices de transformación (modelo, vista y proyección).
- **Cámara y proyección**
  - Cámara fija en tercera persona detrás del lanzador o en primera persona, adjunta al hueso de la cabeza del jugador lanzador.
  - Proyección en perspectiva para dar profundidad a la escena.
- **Sistema de interacción**

La escena permite **dirigir el disparo a cualquier punto visible** de la escena que esté por encima del terreno de juego (césped). Esto no es así cuando el jugador es el portero, que solo puede lanzar a su portero a cualquier punto dentro de la portería. El cursor de apuntado en el rol de jugador se mueve aleatoriamente alrededor del cursor real (ratón o joystick) para añadir dificultad.
- **Transición 2D-3D**

Al iniciar un penalti se pausa la escena 2D se cambia al pipeline de renderizado a OpenGL y se carga la escena 3D. Al finalizar se destruye todo lo referente a la escena 3D (modelos 3D, buffers, etc) y se vuelve al sistema 2D de SFML.

## 9. Organización del proyecto

### 9.1 Tareas y distribución

| Tarea                             | Encargados                                                          |
|-----------------------------------|---------------------------------------------------------------------|
| <b>Partido</b>                    |                                                                     |
| Diseño y movimiento del balón     | Jorge Pérez                                                         |
| Interfaz de usuario               | Nicolás Pascual<br>Enrique Martínez                                 |
| Comportamiento de la IA           | Nicolás Pascual<br>Enrique Martínez                                 |
| Ayudas arbitrales                 | Óscar Jaime<br>Nicolás Pascual                                      |
| Máquina de estados del partido    | Nicolás Pascual<br>Enrique Martínez                                 |
| Mecánicas                         | Nicolás Pascual<br>Enrique Martínez<br>Jorge Pérez<br>Jorge Rodilla |
| Escalado de estadísticas de la IA | Nicolás Pascual<br>Enrique Martínez                                 |
| Minimapa                          | Enrique Martínez                                                    |
| Añadir jugadores extra            | Enrique Martínez<br>Nicolás Pascual                                 |
| Cambios en el sprite del campo    | Enrique Martínez                                                    |
| Intro del partido                 | Enrique Martínez                                                    |
| <b>Roguelike</b>                  |                                                                     |
| Diseño de salas                   | Óscar Jaime                                                         |
| Generación procedural del mapa    | Óscar Jaime                                                         |
| Minimapa                          | Daniel Girón                                                        |
| Sistema de interactivables        | Daniel Girón                                                        |
| Personalización de rivales        | Enrique Martínez<br>Daniel Girón                                    |
| Estadísticas del jugador          | Óscar Jaime                                                         |



|                                              |                                                                    |
|----------------------------------------------|--------------------------------------------------------------------|
| Colisiones                                   | Óscar Jaime                                                        |
| <b>3D</b>                                    |                                                                    |
| Motor 3D modelos estáticos                   | Jorge Rodilla                                                      |
| Animaciones básicas                          | Jorge Rodilla                                                      |
| Sistema de disparo y parada                  | Jorge Rodilla                                                      |
| Ampliación de motor 3D a modelos animados    | Jorge Rodilla                                                      |
| <b>Estructura general</b>                    |                                                                    |
| Movimiento y diseño de sprites               | Jorge Pérez                                                        |
| Arquitectura de salas                        | Daniel Girón                                                       |
| Sistema de guardado                          | Daniel Girón                                                       |
| Gestión del archivo XML                      | Jorge Pérez                                                        |
| Interfaz de usuario                          | Daniel Girón<br>Enrique Martínez                                   |
| Implementación de controles de mando         | Jorge Pérez<br>Enrique Martínez                                    |
| Sonidos y música                             | Daniel Girón<br>Enrique Martínez                                   |
| Comunicación entre Roguelike y partidos      | Daniel Girón<br>Enrique Martínez<br>Óscar Jaime<br>Nicolás Pascual |
| Manager para explicaciones                   | Enrique Martínez<br>Nicolás Pascual                                |
| <b>Arreglos post beta</b>                    |                                                                    |
| Corrección de bugs                           | Nicolás Pascual<br>Enrique Martínez                                |
| Cambios en la dificultad de la IA            | Nicolás Pascual<br>Enrique Martínez                                |
| Separación del menú de sonido                | Enrique Martínez                                                   |
| Acceso rápido a la sala de penalties         | Nicolás Pascual                                                    |
| Creación de un exe portable                  | Nicolás Pascual                                                    |
| Eliminación de la terminal al abrir el juego | Nicolás Pascual                                                    |

Tabla 8. Reparto de tareas

## 9.2 Cronograma

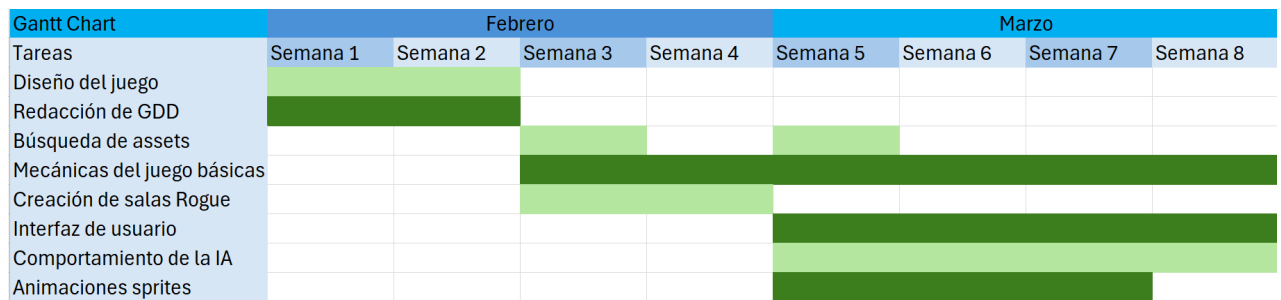


Tabla 9. Descripción del cronograma en los meses de Febrero y Marzo

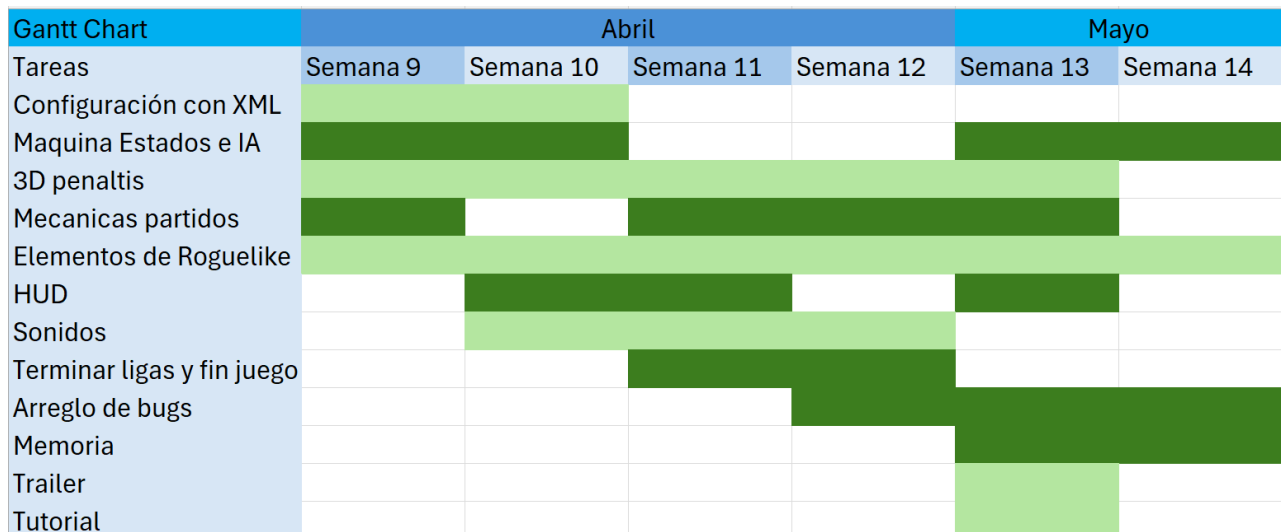


Tabla 10. Descripción del cronograma en los meses de Abril y Mayo

## 10. Problemas encontrados

### 10.1 Falta de assets e inconsistencia del arte

Al tratarse de un concepto de juego original, no existían paquetes de recursos visuales, *assets*, adaptados a nuestras necesidades específicas. Tuvimos que **recopilar y combinar** elementos visuales provenientes de distintos videojuegos.

Esta fragmentación obligó a editar exhaustivamente los sprites existentes, así como rediseñar las pelotas desde cero, y a programar un sistema de máscaras de color por shaders para poder tinter dinámicamente las equipaciones de los diferentes equipos. A pesar de los esfuerzos de homogeneización, el resultado final presenta ciertas inconsistencias visuales, logrando una estética pixel art de 8/16 bits despareja.

### 10.2 Inteligencia artificial (IA) en la simulación de fútbol

Desarrollar una IA competitiva y creíble para un juego de fútbol 4vs4 y escalable a más jugadores resultó ser una tarea de alta complejidad.

Se tuvo que estructurar un **sistema complejo de máquinas de estados** para coordinar comportamientos dinámicos como la presión defensiva por zonas, los roles de cada jugador siendo el delantero, las dos bandas y portero, el cálculo de trayectorias de interceptación, la toma de decisiones para realizar pases y los disparos a puerta. Al recibir retroalimentación tras la versión beta, decidimos simplificar los primeros partidos para que el jugador novato mejorará su experiencia.

### 10.3 Implementación y renderizado 3D en bajo nivel (OpenGL)

Para la escena de la tanda de penaltis decidimos renderizar en 3D. Sin embargo, al utilizar nuestro propio motor basado en la librería SFML, esta carece de soporte nativo para 3D, así que se nos vió obligados a bajar a un nivel de abstracción y programar directamente sobre la **API de OpenGL**, en este caso OpenGL moderno.

Siendo la primera vez que el equipo trabajaba con gráficos 3D en bajo nivel, condensar este desarrollo en un solo mes generó graves dificultades. El pipeline de renderizado manual (gestión de VBOs, VAOs, EBOs, carga de texturas, matrices de proyección y la importación de modelos low-poly con sus respectivas animaciones) requirió un esfuerzo muy superior al estimado inicialmente. Además, las APIs tanto en windows como en linux funcionan de manera distinta y provocaban incompatibilidades y bugs visuales que no se consiguieron arreglar del todo.

### 10.4 Modularidad para modificar número de jugadores

El diseño inicial del juego se concibió inicialmente para partidos con un **número fijo de jugadores 4vs4**. A cada entidad jugador se le asignaron roles y posiciones fijas dentro del código desde las fases tempranas.

Por petición, se pidió que fuera escalable, y por medio del xml se puede cambiar la cantidad de jugadores, aunque no es recomendable para la experiencia de usuario. En una primera instancia **no se contempló que fuese escalable**, así que añadir jugadores provocó que los jugadores controlados por la CPU se amontonaran, tomando decisiones erráticas. Se tuvo que realizar un ajuste en cómo se organizan y mueven los jugadores en las zonas del campo para coexistir más jugadores. Finalmente, aunque es posible, se decidió no añadir más jugadores ya que perjudicaba la experiencia final del usuario sumando dificultad y añadiendo menos control al partido.

## 10.5 Compatibilidad multiplataforma (Linux - Windows)

Garantizar el funcionamiento idéntico del juego tanto en entornos Windows como distribuciones Linux supuso un reto crítico debido a las dependencias de bajo nivel.

La gestión de memoria interna difería entre los compiladores de ambos sistemas operativos, sacando a la luz **punteros colgados o incompatibilidades de falta de inclusión de librerías** que crasheaban el juego, así como gestión de la memoria y de creación de objetos. La carga de las texturas y sprites se realizaban de manera distinta, y provocaba que en el momento de creación de la escena del partido, no se cargase al principio la textura de la máscara. Esto fue debido a un problema crítico de punteros inválidos por el uso de `std::optional` frente a `std::unique_ptr`, sumado a un error en el operador de asignación de copia. Al usar `optional`, el objeto sprite se aloja directamente dentro de la pila del objeto que contiene, en este caso el jugador. Cuando se hacía una asignación directa (`m_sprite = other.m_sprite`) C++ destruía el sprite antiguo y creaba uno nuevo en una dirección de memoria distinta, rompiendo por completo las referencias internas que la textura tenía de él. En windows, donde la gestión de memoria gráfica y de CPU es muy estricta, intentaba pintar un sprite cuya textura no estaba correctamente vinculada en memoria. En Linux, al ser más permisivo con el mapeo de memoria virtual, sobrevivía. Al cambiar a `std::unique_ptr`, el sprite ya no se copia ni se mueve de sitio en la memoria física de forma descontrolada. El objeto reside de manera segura en el heap y lo que se maneja es un puntero inteligente único. Esto garantiza que la dirección de memoria del sprite se mantiene totalmente estable. Además se hace una copia selectiva de los atributos en lugar de reasignación de objetos, que lo que hace es sustituir un objeto sprite entero por otro, destruyendo sus estado interno y sus enlaces de textura, de la otra manera solo clonamos las propiedades físicas y estéticas.

También se encontró **diferencias en el mapeo de inputs y controladores** a nivel de sistema operativo, requiriendo un esfuerzo extra de identificar las asignaciones que tenían cada driver, y permitir la total configuración de estos, incluyendo invertir ejes. Se explica más en profundidad en el siguiente párrafo.

## 10.6 Soporte para todo tipo de controles (Mando y teclado + ratón)

El juego debía ofrecer una experiencia arcade fluida permitiendo el uso de **mandos y teclado + ratón** de manera simultánea e intercambiable.

La gestión de inputs fue un apartado que consumió mucho tiempo por una mala gestión de concurrencia e incompatibilidades. Tuvimos que mapear manualmente diferentes vectores

de entrada (el movimiento digital de las teclas “WASD” frente al rango analógico de los joysticks de los mandos, los cuales se comportan de manera totalmente diferente). Además, tuvimos que implementar zonas muertas y lógicas de prioridad para evitar que el movimiento involuntario o la posición del cursor del ratón interfirieran en la dirección de apuntado cuando el usuario decidía jugar exclusivamente con mando. Conseguir un gameplay pulido y natural para ambos esquemas en un plazo de tiempo ajustado fue costoso también por no haber realizado los controles de teclado a la par que los de mando.

## 10.7 Coordinación en el equipo

El desarrollo se vio afectado negativamente por **problemas internos** de coordinación hicieron que se viese reducido el número de integrantes de 7 a 6, provocando una **reestructuración de tareas y retrasos en la mitad del proyecto**.

Además, coordinarse entre los 6 miembros restantes ha sido todo un reto, para la integración de ramas en períodos de trabajo en paralelo. Se planteó dividir en dos ramas el trabajo, donde cada grupo trabajaba en una, y esto agilizó y consiguió menos conflictos, aunque sin mitigar todos. A través de canales de Discord se mantuvo la comunicación y los cambios que hacía cada integrante además del uso de issues de GitHub para repartirnos las tareas.

## 10.8 Integración de elementos Roguelike en juego de fútbol

Fusionar dos géneros tan dispares como el fútbol arcade y el roguelike supuso un **desafío continuo de diseño de juego**. El bucle principal del juego requería equilibrar de forma justa la progresión de dificultad. Se tuvo que diseñar un sistema preciso de escalado de las estadísticas de la IA mediante multiplicadores por partido y liga. Además, integrar mecánicas puramente aleatorias y de riesgo/recompensa, como los eventos de los ultras en las salas de apuestas, las ayudas arbitrales de un solo uso o la probabilidad de descenso por la intervención de la FIFA en las salas de amañados, requirió un arduo trabajo de testeo y calibración interna para evitar que el juego resultara **frustrante o injusto para el usuario**.

## 11. Cumplimiento de objetivos

En términos generales, el juego final **recoge la gran mayoría de lo planteado en el GDD**, con algunas funcionalidades superadas respecto al diseño original y otras que quedaron fuera por restricciones de tiempo.

Motor, lenguaje y arquitectura se implementaron exactamente como se describió: SFML para todo el juego 2D, OpenGL directo para la escena de penaltis, y C++ con CMake como sistema de build. La transición entre el contexto 2D y 3D funciona mediante el sistema de escenas, reseteando el estado de OpenGL entre cambios.

Las tres ligas y el mapa roguelike se generan de forma procedural tal como se diseñó, con el número de salas y tipos previstos. Todos **los tipos de sala del GDD están presentes**: rival, rival final, cofre, apuestas, amaños y penaltis. Se añadió además una sala de créditos al completar el juego, que no estaba en el diseño original.

El sistema de penaltis 3D es donde el resultado final más **supera al GDD**. El documento planteaba una escena simple con primitivas geométricas básicas; el juego final incorpora modelos animados en formato .glb con skinning, sistema de iluminación y sombras, un cursor tembloroso de apuntado que añade tensión al lanzamiento, y una cámara intercambiable entre tercera y primera persona, esta última no contemplada en el GDD. Además, en el GDD se decidió que solo se podría lanzar en tres direcciones predeterminadas, y lo mismo para defender. Esto se implementó así, pero el profesor pidió que se pudiera apuntar libremente en cualquier dirección, así que se modificó para hacerlo de esta manera.

La economía: oro, puntos de esfuerzo, tienda, entrenamiento, apuestas y amaños se implementó completa, incluyendo el sistema de riesgo de escándalo con porcentajes acumulables y descenso de liga al ser pillado haciendo trampas.

La IA cumple con lo diseñado en líneas generales: máquinas de estados para jugadores de campo y portero, con posicionamiento por zonas y escalado de dificultad por liga. El escalado resultó algo más simplificado que lo descrito en el GDD.

Los controles, HUD y guardado se implementaron según lo previsto. El marcador replica la estética de Nintendo World Cup, el minimapa del roguelike muestra el progreso por salas, y el guardado automático persiste el estado completo en cada transición relevante.

Lo que quedó fuera fueron: la ampliación de jugadores para los partidos finales de cada liga, las **reglas especiales de partido** (ganar con diferencia de goles, mantener ventaja durante un tiempo) y **las tarjetas rojas**, ambas descritas en el GDD pero no implementadas en la versión final.

## 12. Mejoras después de la beta

Tras la beta hubo muchos comentarios sobre la **alta dificultad** del juego lo que provocaba una sensación de mala jugabilidad y mala implementación de la IA en los partidos, debido a ello se mejoraron varios aspectos al respecto.

- En el estado “perseguir balón”, ahora solo el jugador más cercano va a por él, evitando provocar que haya tantos jugadores juntos en un mismo punto del campo.
- También se creó una pequeña ventana de tiempo donde la IA no te hace una entrada si se acaba de realizar una acción, como un pase, para tener tiempo a realizar una acción y que no te quite el balón instantáneamente.
- Respecto a la dificultad también se creó **un nivel de IA más simple**, para los dos primeros partidos, donde la IA en estado defensivo no realiza presión.

Con estos cambios se ha producido un cambio en la curva de aprendizaje y ahora **el inicio es más asequible** para jugadores nuevos, además se han reducido drásticamente las situaciones de cúmulos de entradas seguidas.

Se han mejorado los **menús diferenciando el nivel de volumen de música y efectos**, además de correcciones visuales y bugs en alguno de los menús.

Se han corregido la mayoría de los **bugs** conocidos referidos al control del juego usando un mando de **PlayStation 4**, tanto en partido como en los penaltis.

Ahora si pierdes un partido y vuelves a la sala inicial, al entrar en la liga de nuevo, entras al mismo mapa de antes guardando el estado de todo lo que habías hecho y pudiendo volver a enfrentarte al rival que te ha ganado.

Se han corregido los **problemas de retorno de la escena de penaltis** hacia cualquier otra. Aun así se ha detectado que en algún ordenador los penaltis fallan sin motivo aparente mientras que en otros funcionan bien.

Se han **corregido distintos bugs** durante casos concretos que se daban en la ejecución de los partidos.

Por último, en la beta se envió un instalador del juego, cosa que no resultó agradable a algunos usuarios. Ahora generamos .zip que contiene: un **.exe portable con el juego**, el fichero **config.xml**, la carpeta de **assets** y **dos ficheros .ini** generados al iniciar el juego.



## 13. Conclusión y cosas a futuro

El desarrollo de *Ascension* ha representado todo un reto de ingeniería en equipo. Se han **cumplido con éxito** los objetivos principales del proyecto, logrando una base robusta y funcional que integra de manera sólida las mecánicas arcade de un juego de fútbol con la progresión y aleatoriedad características del género *roguelike*. El software es capaz de gestionar la persistencia de estados, simulación deportiva mediante inteligencia artificial, la generación procedural de eventos y de escenarios y el manejo básico del 3D.

Con el fin de evolucionar el prototipo actual hacia una versión competitiva, se proponen las siguientes mejoras para el futuro:

- **Escalabilidad, jugabilidad e IA:** Perfeccionar los algoritmos de toma de decisiones de la CPU para dotar a los contrincantes de un juego más orgánico y fluido. Añadiendo una experiencia distinta al usuario y un mayor reto.
- **Cinemáticas:** Integrar cinemáticas, por ejemplo una introductoria al inicio de la partida para contextualizar la narrativa del club de fútbol en su ascenso hacia la élite, mejorando la inmersión del usuario.
- **Apartado gráfico:** Consolidar el renderizado 3D, depurando por completo bugs visuales remanentes y optimizando la carga de shaders. Adicionalmente, añadir más animaciones a los modelos y enriquecer la escena con más técnicas de sombreados y texturas. Por último, rediseñar el arte 2D y estandarizar los sprites. El objetivo es solventar las inconsistencias heredadas del uso de diferentes paquetes de recursos y lograr una estética pixel-art limpia y uniforme en todo el juego. Además de dotar a la parte del roguelike de más decoración y más eventos para aportar algo más de vivacidad a las ligas.
- **Testing y balanceo:** Llevar a cabo intensas sesiones de *playtesting* orientadas a la recogida de datos estadísticos. Esto permitirá equilibrar y testear de forma masiva la economía de las salas, la tasa de apariciones de ciertos objetos, modificadores arbitrales y ventajas competitivas para asegurar que el juego sea justo, desafiante y adictivo. De esta forma también mejoraría la curva de dificultad del juego.
- **Apartado sonoro:** Sustituir los recursos de audio genéricos o con copyright por una composición musical propia e interactiva, adaptada dinámicamente a la tensión de los partidos y al objetivo del juego.